Prowider Mini Lead Distribution System

Internship Assessment Submission Documentation

---

1. Project Overview

This project is a simplified lead distribution platform inspired by real-world systems like Prowider. Customers can submit service requests through a public form, and the system automatically distributes leads to providers according to predefined business rules.

The implementation includes:
• PostgreSQL database persistence
• Fair provider allocation logic
• Concurrency-safe transactions
• Real-time dashboard updates
• Webhook idempotency handling
• Quota enforcement
• Duplicate lead prevention

2. Tech Stack Used

Frontend:
• Next.js 15
• React
• TypeScript
• TailwindCSS

Backend:
• Next.js API Routes
• Prisma ORM
• Socket.IO

Database:
• PostgreSQL

3. Setup Instructions

Step 1 — Install Dependencies
npm install

Step 2 — Configure Environment Variables
Create a .env file in the root directory and add:
DATABASE_URL="postgresql://postgres:PASSWORD@localhost:5432/prowider"

Step 3 — Generate Prisma Client
npx prisma generate

Step 4 — Run Database Migration
npx prisma migrate dev --name init

Step 5 — Seed Initial Data
npm run seed

Step 6 — Start Development Server

npm run dev

Available Routes:
• /request-service
• /dashboard
• /test-tools

4. Allocation Algorithm

The allocation system follows strict business rules to ensure fairness and correctness.

1. A lead is first stored in the database.
2. Mandatory providers are assigned depending on the service type.
3. Remaining provider slots are filled using a persistent round-robin algorithm.
4. Allocation state is stored inside the database using an AllocationState table.
5. Providers exceeding monthly quota are skipped automatically.
6. Each lead is assigned to exactly 3 providers.

This guarantees:
• Fair distribution
• Persistent rotation
• No provider favoritism
• Correct quota handling

5. Concurrency Handling

Concurrency handling was implemented using:
• Prisma database transactions
• Serializable transaction isolation level

All lead creation and provider allocation logic executes inside a single database transaction. This prevents:
• Race conditions
• Double assignment
• Incorrect quota counts
• Corrupted allocation state

6. Webhook Idempotency

A webhook simulation endpoint was implemented for resetting provider quotas.

To ensure idempotency:
• Every webhook request contains an idempotency key.
• Processed webhook keys are stored in the WebhookEvent table.

Before processing, the system checks whether the idempotency key already exists. If already processed:
• The webhook effect is skipped
• Quotas are not reset again
• Duplicate processing is prevented

7. Real-Time Dashboard

Real-time updates were implemented using Socket.IO.

When a new lead is created:
• The backend emits a socket event.
• Connected dashboard clients automatically refresh their data.

This allows providers to instantly see newly assigned leads without manually refreshing the page.

8. Conclusion

This project focuses heavily on backend reliability, engineering correctness, database consistency, and concurrency safety.

The implementation satisfies all major assignment requirements including:
• Correct provider allocation
• Fair round-robin distribution
• Persistent allocation state
• Duplicate lead prevention
• Real-time updates
• Concurrency-safe logic
• Webhook idempotency